

○×ゲームロボットシステム: Masterモジュール開発 要件定義書

1. プロジェクト概要

SO-100系アームロボット、カメラ(盤面検出)、Unity上の3Dキャラクター(Agent)を連携させ、人間とAIが○×ゲーム(三目並べ)を行うシステムにおける、「Masterモジュール」の開発依頼です。

Masterモジュールは、ゲーム全体の進行管理、Pythonロジックによるゲームルール(勝敗)の判定、外部LLM APIを用いた推論、および各モジュール(Robot, Vision, Unity)との非同期通信・制御を行うシステムの「頭脳」となります。

2. 業務スコープ

2.1 対象範囲(インスコープ)

- **Masterモジュール**のアプリケーション開発(※**Python 3.x** 必須)。
- **WebSocket**サーバーの構築(Robot, Vision, Unityなどのクライアントからの接続を非同期で管理)。
- ゲームルールのロジック実装(勝敗・引き分け判定、および「人間の有効な手」のバリデーション)。
- 非同期ポーリングロジックの実装(人間のターンにおける定周期の盤面監視と安定判定)。
- **LLM API**との連携処理、エラーリカバリー(フォールバック)の実装。
- システム制御・監視用**API (Start / Reset / 内部状態取得)**の実装。
- テストコード / モックプログラムの作成。

2.2 対象外(アウトオブスコープ)

- Robotモジュール(実機のアーム制御)、Visionモジュール(画像認識)、Unityモジュールの実装。
- 実機を用いたシステム全体の結合テスト。

3. 機能要件

3.1 ゲーム進行管理とステートマシン

システムは以下の状態を持ち、適切に遷移・制御を行うこと。

※勝敗判定や「有効な手」の判定は、**Master**側の**Python**ロジックで厳密に行うこと。**Master**が保持する盤面データがゲームの「正」となります。

1. 待機(**Standby**):
 - システム起動時の初期状態。外部からの Start 要求を待機。
2. ゲーム開始:
 - Start 要求(先手・後手指定含む)を受信し、指定ターンへ移行。
3. 人間のターン(ポーリング監視):
 - 1秒おきにVisionモジュールへ盤面要求(request_board_state)を送信。
 - 受信した盤面が「現在のMasterの内部状態から、人間(1)が空きマスに1つだけ追加された状態(有効な手)」であるかを判定。
 - 有効な手で更新されており、かつその同一の盤面状態が2回連続で確認できた場合(安定状態)、人間の手が確定したとみなし、Masterの内部盤面を更新する。
 - ※途中で盤面が変化した場合(手が迷っている等)はカウントをリセットし、監視を継続する。
 - 確定後、勝敗判定を行い、継続なら「AIのターン」へ移行。勝敗が決定した場合は勝敗決着時の挙動とリセットフローへ移行(詳細は3.2参照)
4. AIのターン(推論と指示):
 - Unityに思考状態(state: thinking)を通知。
 - Masterの最新内部盤面データをLLM APIに送信し、「次の一手」「感情」「セリフ」を取得(詳細は3.3参照)。
 - Unityへ感情とセリフ(play_reaction)を送信。
 - Robotへコマ配置(place_piece)を指示。
5. AIの手の実行と検証:
 - Robotからの配置完了(placement_result: success=true)を受信。
 - 直後に**Vision**モジュールへ盤面要求を送信し、盤面をチェックする。
 - 正常(整合性あり): Masterが把握している「AIが置いた後の想定盤面」とVisionの認識が一致すれば、Masterの内部盤面を更新し、勝敗判定を行う。継続なら「人間のターン」へ。勝敗が決定した場合は勝敗決着時の挙動とリセットフローへ移行(詳細は3.2参照)
 - 異常(不整合あり): 意図しない盤面になっている、またはRobotから配置失敗(success=false)が返ってきた場合は即座にエラーとする。Unityへエラー演出を送信し、内部状態を待機(Standby)へ戻し、Robotへリセット(reset_robot)を送信。
6. 強制リセット(随時):
 - Reset 要求を受け付けた場合、進行中の処理を中断し、初期化。各クライアントへリセットを通知する。

3.2 勝敗決着時の挙動とリセットフロー

ゲームの決着(AI勝利、人間勝利、または引き分け)をPythonロジックが検知した場合、以下の手順

を実行する。

1. 結果判定と通知:
 - 検知した結果に基づき、Unityへ演出指示を送信する。
 - LLMIには「ゲーム結果」と「それに応じた結びのセリフ」を生成させる。
2. 終了待機(演出時間の確保):
 - 演出完了および人間が盤面を確認する時間を確保するため、5秒の待機時間を設ける。
3. 自動リセットの実行:
 - 待機時間終了後、Masterは自動的にリセット処理を実行する。
 - Robotへreset_robot(初期位置への復帰)を送信。
 - Unityへset_state: idleを送信。
 - Master内部の盤面配列をすべて0にクリアし、ステートをStandbyへ遷移させる

3.3 LLM API連携およびエラーリカバリー仕様

- 使用モデルの指定: 外部LLMとして、OpenAI API(gpt-4o または gpt-4o-mini)などを想定する。(※開発者が選定してよい)
- 要求する出力フォーマット: JSONモード(またはStructured Outputs)を指定し、以下のスキーマでの返答を強制する。

```
{  
  "next_move": 4,           // 0~8の整数。配置するマスのインデックス  
  "emotion": "joy",         // "joy", "sorrow", "angry", "fun", "neutral" 等の指定された感情文字列  
  "dialogue": "そこですか！" // Unityでエージェントに発話させるセリフ  
}
```

- **LLMのエラーリカバリーと上限回数:**
LLMの返答がJSONフォーマット違反である場合、または next_move が「既にコマが置かれているマス」や「範囲外(0~8以外)」であった場合、現在の盤面状態とエラー内容を付与して再リクエスト(リトライ)を行う。
 - 最大リトライ回数: 3回とする。
- **フォールバック処理:**
3回リトライしても有効な手が得られなかった場合は、安全稼働を優先し以下の値を出力として採用し、ゲームを続行する。
 - next_move: 空いているマスからランダムに選択
 - emotion: "neutral"
 - dialogue: 定型文(エラー発生を悟らせない自然なセリフ。例:「なるほど、ではこちらにしましょう」等)

4. 通信インターフェース仕様

Masterはサーバーとして稼働します。通信プロトコルの基本はWebSocket(JSON形式)とします。

4.1 盤面配列(0~8)の物理的な位置マッピング定義

Master、Vision、Robot、Unity間で座標認識のズレを防ぐため、盤面配列 [0, 1, 2, 3, 4, 5, 6, 7, 8] の各インデックスは、「人間側正面から見て」以下のようにマッピングされるものと定義する。

【盤面の物理マッピング】

[0][1][2]

[3][4][5]

[6][7][8]

※配列の値について: 0 = 空き, 1 = 人間のコマ(O), 2 = AIのコマ(X) とする。

4.2 対 Vision モジュール(盤面検出)

- 【送信】盤面要求: {"type": "request_board_state", "payload": {}}
- 【受信】盤面返答: {"type": "board_state_response", "payload": {"board": [1, 0, 2, 0, 1, 0, 0, 0, 0]}}

4.3 対 Robot モジュール(物理駆動)

- 【送信】配置指示: {"type": "place_piece", "payload": {"position": 4, "piece_type": 2}} (※LLMの next_move の値を position として渡す)
- 【受信】配置結果: {"type": "placement_result", "payload": {"success": true, "position": 4, "error_detail": null}}
- 【送信】リセット: {"type": "reset_robot", "payload": {}}

4.4 対 Unity モジュール(演出)

- 【送信】状態変更: {"type": "set_state", "payload": {"state": "thinking"}}
- 【送信】演出指示: {"type": "play_reaction", "payload": {"emotion": "joy", "dialogue": "そこですか!"}}

4.5 システム制御・監視用 API (Start / Reset / State)

※WebSocketの新規メッセージ、または別途HTTP(FastAPI等)のREST APIとして実装。

- 【受信】ゲーム開始: {"type": "start_game", "payload": {"first_turn": "human"}}
- 【受信】強制リセット: {"type": "force_reset", "payload": {}}
- 【受信】内部状態の取得: {"type": "get_internal_state", "payload": {}}
- 【送信】内部状態の返答: {"type": "internal_state_response", "payload": {"board":

```
[1,0,2,0,1,0,0,0,0], "current_phase": "human_turn"]}}
```

4.6 異常系ハンドリング(タイムアウトと切断)

システムの安定稼働のため、各非同期通信に対して以下のタイムアウトおよび切断検知を実装すること。

- 各リクエストのタイムアウト定義:
 - **Vision**モジュールへの盤面要求: 1秒以内に応答がない場合はタイムアウトとし、再度リクエストを送信する(最大3回)。
 - **LLM API**への推論要求: 10秒以内に応答がない場合はタイムアウトとし、リトライする(最大3回)。それでも失敗した場合は3.2のフォールバック処理を適用する。
 - **Robot**モジュールへの配置指示: 物理的な動作を伴うため長めに設定し、30秒以内に{"success": true}(またはfalse等の応答)がない場合はロボット異常とみなし、システム全体を「エラー停止状態(リセット処理)」へ遷移させる。
- **WebSocket**切断時の挙動:
対局中に Robot, Vision, Unity のいずれかのクライアントの接続が切断された場合、直ちにエラーとみなし、可能な限りシステム全体のリセット(待機状態への初期化)を行うこと。

5. テスト・品質保証要件(必須)

他モジュールを模倣したPython製のモッククライアントを作成し、以下のシナリオが正常に動作することを証明してください。

必須検証シナリオ

1. 正常系
 - クライアントが正常にレスポンスを返した場合に、ゲームが適切に進行され、勝敗確定時に待機状態に遷移すること。
2. 人間のターン: ポーリングと安定判定の検証
 - 1秒おきの盤面要求において、「有効な手が2回連続で返ってきた時のみ」手が確定し、進行すること。
3. AIのターン: 配置後の盤面検証とエラーハンドリング
 - モックRobotが success: true を返した後、モックVisionが「AIが意図した手と異なる盤面」を返した場合、即座にエラーハンドリングが行われること。
 - モックRobotが success: falseを返した後、即座にエラーハンドリングが行われること。
4. LLMのエラーリカバリーとフォールバックの検証
 - LLMモックが「既に埋まっているマス」を返した際、再リクエストが行われること。
 - 3回連続で無効なレスポンスを返した際、ランダムな空きマスと定型文のフォールバックが適用され、ゲームがクラッシュせずに続行すること。
5. タイムアウト・切断処理の検証
 - モックRobotの応答を30秒以上遅延させた際、Masterが正しくエラー状態へ移行・リセット

処理を行うこと。

- 強制リセット要求時に、Masterが正しくエラー状態へ移行・リセット処理を行うこと。
- モッククライアントが切断された際に、Masterが正しくエラー状態へ移行・リセット処理を行うこと。

※納品時には、pytest 等を用いた自動テストコード、またはテスト手順書とモックスクリプトを提出してください。

6. 開発環境・支給要件

- 開発言語: **Python 3.x** 必須。asyncio による非同期タスク制御を適切に実装すること。
- **APIキー**: LLM利用に必要なAPIキーは当方から支給します。

※Unity側の詳細仕様(とりうる感情状態など)は後日担当者から連携されます。